JAVA STREAMS & ALGORITHMS — Complete Program Library

STREAM METHOD NAMES — Read Like English

Examples That Read as Natural Sentences

Pattern 1: Filter → Transform → Collect

```
// "From list, keep adults, get their names, collect to list"
list.stream().filter(Person::isAdult).map(Person::getName).collect(toList());

// "From products, keep in stock, get prices, collect to set"
products.stream().filter(Product::isInStock).map(Product::getPrice).collect(toSet(
));

// "From employees, keep IT department, get salaries, collect to array"
employees.stream().filter(e ->
"IT".equals(e.getDept())).map(Employee::getSalary).toArray();

// "From orders, keep completed, get amounts, collect to map"
orders.stream().filter(Order::isCompleted).collect(toMap(Order::getId,
Order::getAmount));
```

Pattern 2: Find First/Any Matching

```
// "From users, find first active user"
users.stream().filter(User::isActive).findFirst();

// "From numbers, find any even number"
numbers.stream().filter(n -> n % 2 == 0).findAny();

// "From strings, find first longer than 5 characters"
strings.stream().filter(s -> s.length() > 5).findFirst();

// "From products, find any with price less than 100"
products.stream().filter(p -> p.getPrice() < 100).findAny();
```

Pattern 3: Check Conditions (Any/All/None)

```
// "From list, check if any price is over 1000"
prices.stream().anyMatch(p -> p > 1000);

// "From students, check if all passed"
students.stream().allMatch(Student::hasPassed);

// "From strings, check if none are empty"
strings.stream().noneMatch(String::isEmpty);

// "From employees, check if all have valid email"
employees.stream().allMatch(e -> e.getEmail().contains("@"));
```

Pattern 4: Count Elements

```
// "From list, count active users"
users.stream().filter(User::isActive).count();

// "From orders, count completed orders"
orders.stream().filter(Order::isCompleted).count();

// "From products, count out of stock items"
products.stream().filter(p -> p.getStock() == 0).count();

// "From sentences, count words longer than 7 letters"
words.stream().filter(w -> w.length() > 7).count();
```

Pattern 5: Sum and Average

```
// "From salaries, sum all values"
salaries.stream().mapToDouble(Double::doubleValue).sum();

// "From ages, calculate average"
ages.stream().mapToInt(Integer::intValue).average();

// "From transactions, sum amounts"
transactions.stream().mapToDouble(Transaction::getAmount).sum();

// "From scores, get average score"
scores.stream().mapToInt(Integer::intValue).average().orElse(0);
```

Pattern 6: Find Min and Max

```
// "From products, find cheapest"
products.stream().min(Comparator.comparing(Product::getPrice));

// "From employees, find highest paid"
employees.stream().max(Comparator.comparing(Employee::getSalary));

// "From dates, find earliest"
dates.stream().min(Comparator.naturalOrder());

// "From numbers, find maximum"
numbers.stream().max(Integer::compareTo);
```

Pattern 7: Group By Something

```
// "From employees, group by department"
employees.stream().collect(groupingBy(Employee::getDepartment));

// "From orders, group by status"
orders.stream().collect(groupingBy(Order::getStatus));

// "From products, group by category"
products.stream().collect(groupingBy(Product::getCategory));

// "From students, group by grade"
students.stream().collect(groupingBy(Student::getGrade));
```

Pattern 8: Group and Count

```
// "From employees, group by department and count each"
employees.stream().collect(groupingBy(Employee::getDepartment, counting()));

// "From orders, group by status and count"
orders.stream().collect(groupingBy(Order::getStatus, counting()));

// "From words, group by first letter and count"
words.stream().collect(groupingBy(w -> w.charAt(0), counting()));

// "From logs, group by level and count"
logs.stream().collect(groupingBy(Log::getLevel, counting()));
```

Pattern 9: Convert List to Map

```
// "From employees, convert to map by id"
employees.stream().collect(toMap(Employee::getId, Function.identity()));

// "From products, convert to map by name"
products.stream().collect(toMap(Product::getName, Function.identity()));

// "From users, convert to map by email"
users.stream().collect(toMap(User::getEmail, Function.identity()));

// "From accounts, convert to map by account number"
accounts.stream().collect(toMap(Account::getNumber, Function.identity()));
```

Pattern 10: Join Strings

```
// "From names, join with comma"
names.stream().collect(joining(", "));

// "From words, join with space"
words.stream().collect(joining(" "));

// "From ids, join with hyphen"
ids.stream().collect(joining("-"));

// "From lines, join with newline"
lines.stream().collect(joining("\n"));
```

Pattern 11: Sort Elements

```
// "From employees, sort by salary"
employees.stream().sorted(comparing(Employee::getSalary));

// "From products, sort by price descending"
products.stream().sorted(comparing(Product::getPrice).reversed());

// "From names, sort alphabetically"
names.stream().sorted();

// "From dates, sort oldest first"
dates.stream().sorted();
```

Pattern 12: Limit and Skip

```
// "From list, take top 10"
items.stream().limit(10);

// "From results, skip first 5"
results.stream().skip(5);

// "From sales, take best 3"
sales.stream().sorted(comparing(Sale::getAmount).reversed()).limit(3);

// "From logs, skip first 100 lines"
logs.stream().skip(100);
```

Pattern 13: Remove Duplicates

```
// "From list, get unique names"
names.stream().distinct();

// "From numbers, get unique values"
numbers.stream().distinct();

// "From emails, remove duplicates"
emails.stream().distinct();

// "From ids, get unique ids"
ids.stream().distinct();
```

Pattern 14: Flatten Nested Structures

```
// "From list of lists, flatten to single list"
listOfLists.stream().flatMap(List::stream);

// "From sentences, get all words"
sentences.stream().flatMap(s -> Arrays.stream(s.split(" ")));

// "From orders, get all items"
orders.stream().flatMap(o -> o.getItems().stream());

// "From customers, get all phone numbers"
customers.stream().flatMap(c -> c.getPhones().stream());
```

Pattern 15: Peek (Debug)

```
// "From stream, log each element then collect"
stream.peek(System.out::println).collect(toList());

// "From orders, log processing then filter"
orders.stream().peek(o -> log.info("Processing: {}",
o)).filter(Order::isValid).collect(toList());

// "From users, log before and after transformation"
users.stream().peek(u -> log.debug("Before: {}", u)).map(User::normalize).peek(u -
> log.debug("After: {}", u)).collect(toList());
```

Pattern 16: Reduce to Single Value

```
// "From numbers, sum all using reduce"
numbers.stream().reduce(0, Integer::sum);

// "From strings, concatenate all"
strings.stream().reduce("", String::concat);

// "From products, find most expensive"
products.stream().reduce((p1, p2) -> p1.getPrice() > p2.getPrice() ? p1 : p2);

// "From transactions, calculate total with initial value"
transactions.stream().reduce(BigDecimal.ZERO, BigDecimal::add, BigDecimal::add);
```

Pattern 17: Partition Into Two Groups

```
// "From employees, partition by salary > 50000"
employees.stream().collect(partitioningBy(e -> e.getSalary() > 50000));

// "From orders, partition by completed"
orders.stream().collect(partitioningBy(Order::isCompleted));

// "From numbers, partition by even or odd"
numbers.stream().collect(partitioningBy(n -> n % 2 == 0));

// "From students, partition by pass or fail"
students.stream().collect(partitioningBy(Student::hasPassed));
```

Pattern 18: Get Summary Statistics

```
// "From ages, get statistics"
ages.stream().mapToInt(Integer::intValue).summaryStatistics();

// "From salaries, get stats"
salaries.stream().mapToDouble(Double::doubleValue).summaryStatistics();

// "From scores, get all stats at once"
scores.stream().mapToInt(Integer::intValue).summaryStatistics();

// "From transaction amounts, get summary"
transactions.stream().mapToDouble(Transaction::getAmount).summaryStatistics();
```

Pattern 19: Handle Nulls

```
// "From list, filter out nulls then process"
list.stream().filter(Objects::nonNull).map(String::toUpperCase).collect(toList());

// "From values, use orElse for default"
stream.findFirst().orElse(defaultValue);

// "From optional, get or throw"
optional.orElseThrow(() -> new NotFoundException());

// "From results, provide fallback value"
results.stream().findAny().orElseGet(() -> createDefault());
```

Pattern 20: Parallel Processing

```
// "From large list, process in parallel"
largeList.parallelStream().filter(Item::isValid).collect(toList());

// "From employees, calculate sum in parallel"
employees.parallelStream().mapToDouble(Employee::getSalary).sum();

// "From orders, group in parallel"
orders.parallelStream().collect(groupingByConcurrent(Order::getStatus));

// "From transactions, process fastest way"
transactions.parallelStream().map(Transaction::process).collect(toList());
```

Pattern 21: Combine Multiple Conditions

```
// "From products, keep in stock AND price < 100"
products.stream().filter(p -> p.isInStock() && p.getPrice() <
100).collect(toList());

// "From employees, keep IT department AND salary > 80000"
employees.stream().filter(e -> "IT".equals(e.getDept()) && e.getSalary() >
80000).collect(toList());

// "From orders, keep completed AND amount > 1000"
orders.stream().filter(o -> o.isCompleted() && o.getAmount() >
1000).collect(toList());

// "From students, keep age > 18 AND grade A"
students.stream().filter(s -> s.getAge() > 18 &&
"A".equals(s.getGrade())).collect(toList());
```

Pattern 22: Chain Transformations

```
// "From list, filter, then transform, then collect"
list.stream().filter(Item::isActive).map(Item::getName).map(String::toUpperCase).c
ollect(toList());

// "From orders, filter completed, get amounts, sum them"
orders.stream().filter(Order::isCompleted).map(Order::getAmount).reduce(BigDecimal
.ZERO, BigDecimal::add);

// "From employees, keep seniors, get emails, collect to set"
employees.stream().filter(e -> e.getYears() >
10).map(Employee::getEmail).collect(toSet());

// "From products, apply discount, filter by price, get names"
products.stream().map(p -> p.withDiscount(0.1)).filter(p -> p.getPrice() <
50).map(Product::getName).collect(toList());
```

Pattern 23: Custom Comparisons

```
// "From employees, sort by salary, then by name"
employees.stream().sorted(comparing(Employee::getSalary).thenComparing(Employee::g
etName)).collect(toList());

// "From products, sort by category, then by price"
products.stream().sorted(comparing(Product::getCategory).thenComparing(Product::ge
tPrice)).collect(toList());

// "From students, sort by grade descending, then by name"
students.stream().sorted(comparing(Student::getGrade).reversed().thenComparing(Stu
dent::getName)).collect(toList());

// "From tasks, sort by priority, then by due date"
tasks.stream().sorted(comparing(Task::getPriority).thenComparing(Task::getDueDate)
).collect(toList());
```

Pattern 24: Collect to Specific Collection Type

```
// "From items, collect to linked list"
items.stream().collect(toCollection(LinkedList::new));

// "From keys, collect to tree set"
keys.stream().collect(toCollection(TreeSet::new));

// "From values, collect to array list with capacity"
values.stream().collect(toCollection(() -> new ArrayList<>(100)));

// "From unique items, collect to concurrent set"
items.stream().collect(toCollection(ConcurrentSkipListSet::new));
```

Pattern 25: Handle Empty Results Gracefully

```
// "From list, find first or return null"
list.stream().filter(Item::isValid).findFirst().orElse(null);

// "From results, get value or compute default"
stream.findAny().orElseGet(() -> createFallbackValue());

// "From search, get or throw meaningful exception"
searchResults.stream().findFirst().orElseThrow(() -> new
NoSuchElementException("Not found"));

// "From empty stream, return empty list safely"
emptyStream.collect(toList()); // Just works, returns empty list
```

Memory Trick: Read as English Sentence

```
Source     + Operation Chain                    + Terminal
_____________________________________________________________

"From list"  "keep adults"  "get names"  "collect to list"
"From users" "find first"   "active"     "or return null"
"From orders" "group by"    "status"     "count each"
"From numbers" "check if"   "all positive"
"From products" "find"      "cheapest"
"From words"   "join with"  "comma"
"From employees" "sort by"  "salary"     "take top 5"
```

Pro Tip: Method Names Are Verbs

Method	Think of it as
`filter()`	"keep only..."
`map()`	"convert each..."
`flatMap()`	"explode each into..."
`sorted()`	"arrange in order..."
`limit()`	"take first..."
`skip()`	"ignore first..."

`distinct()`	"remove duplicates..."
`peek()`	"look at each..."
`collect()`	"gather into..."
`groupingBy()`	"put into buckets by..."
`partitioningBy()`	"split into two groups..."
`joining()`	"glue together with..."
`findFirst()`	"get the first one..."
`anyMatch()`	"is there any..."
`allMatch()`	"do all..."
`noneMatch()`	"is there no..."

JAVA 8 STREAMS — Pure Methods Reference Table

No Code — Only Method Names & When to Use

---

1. STREAM CREATION METHODS

Method	When to Use
`stream()`	Convert collection to stream
`of()`	Create stream from individual elements
`iterate()`	Generate infinite sequential stream
`generate()`	Create infinite stream from supplier
`range()`	Create numeric range (exclusive end)
`rangeClosed()`	Create numeric range (inclusive end)
`concat()`	Merge two streams together
`empty()`	Create empty stream

`builder()`	Build stream dynamically
`chars()`	Convert string to IntStream

---

2. INTERMEDIATE OPERATIONS (Transform)

Filtering Methods

Method	When to Use
`filter()`	Keep elements matching condition
`distinct()`	Remove duplicates
`limit()`	Take first N elements
`skip()`	Discard first N elements
`takeWhile()`	Keep until condition fails (Java 9+)
`dropWhile()`	Discard until condition fails (Java 9+)

Transformation Methods

Method	When to Use
`map()`	Convert each element 1→1
`flatMap()`	Convert each element 1→many
`mapToInt()`	Convert to IntStream
`mapToLong()`	Convert to LongStream
`mapToDouble()`	Convert to DoubleStream
`flatMapToInt()`	Flatten to IntStream
`boxed()`	Convert primitive stream to object stream

Sorting Methods

Method	When to Use
`sorted()`	Sort naturally

`sorted(Comparator)`	Sort with custom rule

Debugging Method

Method	When to Use
`peek()`	Debug - see intermediate values

---

3. TERMINAL OPERATIONS (Execute)

Iteration Method

Method	When to Use
`forEach()`	Perform action on each element
`forEachOrdered()`	Preserve order in parallel streams

Aggregation Methods

Method	When to Use
`count()`	Count total elements
`min()`	Find smallest element
`max()`	Find largest element
`sum()`	Add all numbers (primitive streams)
`average()`	Calculate mean (primitive streams)
`reduce()`	Combine all to one value

Matching Methods

Method	When to Use
`anyMatch()`	Check if ANY element matches
`allMatch()`	Check if ALL elements match
`noneMatch()`	Check if NO elements match

Finding Methods

Method	When to Use
`findFirst()`	Get first element (ordered)
`findAny()`	Get any element (parallel friendly)

Collection Methods

Method	When to Use
`collect()`	Gather results into container
`toList()`	Simplified collection (Java 16+)
`toArray()`	Convert to array

---

4. COLLECTORS METHODS (Inside collect)

Grouping Methods

Method	When to Use
`groupingBy()`	Group elements by key
`groupingByConcurrent()`	Thread-safe grouping
`partitioningBy()`	Split into true/false groups

Joining Methods

Method	When to Use
`joining()`	Combine strings
`joining(delimiter)`	Join with separator
`joining(delimiter, prefix, suffix)`	Join with wrapper

Summarizing Methods

Method	When to Use
`summarizingInt()`	Get all stats (count, sum, min, max, avg)

`summarizingLong()`	Same for long values
`summarizingDouble()`	Same for double values

Mapping & Reducing

Method	When to Use
`mapping()`	Transform before collecting
`flatMapping()`	Flatten then collect
`filtering()`	Filter before collecting
`reducing()`	Custom reduction
`collectingAndThen()`	Post-process result

To Collection Methods

Method	When to Use
`toList()`	Collect to ArrayList
`toSet()`	Collect to HashSet
`toCollection()`	Collect to specific collection type
`toMap()`	Convert to Map
`toConcurrentMap()`	Thread-safe Map
`toUnmodifiableList()`	Create immutable list (Java 10+)

Counting & Averaging

Method	When to Use
`counting()`	Count elements in group
`averagingInt()`	Average of ints
`averagingDouble()`	Average of doubles
`summingInt()`	Sum of ints

---

5. PRIMITIVE STREAM METHODS

IntStream Methods

Method	When to Use
`range()`	Create sequence start to end-1
`rangeClosed()`	Create sequence start to end
`sum()`	Add all values
`average()`	Calculate mean
`summaryStatistics()`	Get all stats at once
`boxed()`	Convert to Stream

DoubleStream Methods

Method	When to Use
`of()`	Create from double values
`sum()`	Add all values
`average()`	Calculate mean

LongStream Methods

Method	When to Use
`range()`	Sequence of longs
`sum()`	Add all values

---

6. PARALLEL STREAM METHODS

Method	When to Use
`parallelStream()`	Convert collection to parallel stream
`parallel()`	Convert sequential to parallel
`sequential()`	Convert parallel to sequential

`isParallel()`	Check if stream is parallel
`unordered()`	Remove order constraint (improves performance)

---

7. OPTIONAL METHODS (Stream Results)

Method	When to Use
`get()`	Get value (unsafe - only if present)
`orElse()`	Provide default value
`orElseGet()`	Provide default from supplier
`orElseThrow()`	Throw exception if empty
`ifPresent()`	Execute action if present
`isPresent()`	Check if value exists
`isEmpty()`	Check if empty (Java 11+)
`filter()`	Apply condition to Optional
`map()`	Transform Optional value
`flatMap()`	Flatten nested Optional
`stream()`	Convert Optional to Stream

---

8. COMPARATOR METHODS (For Sorting)

Method	When to Use
`comparing()`	Sort by field
`comparingInt()`	Sort by int field
`comparingDouble()`	Sort by double field
`thenComparing()`	Secondary sort
`reversed()`	Reverse order

`naturalOrder()`	Default natural order
`reverseOrder()`	Reverse natural order
`nullsFirst()`	Put nulls at beginning
`nullsLast()`	Put nulls at end

---

9. QUICK DECISION MATRIX

I Need To...	Use Method
Start streaming	`stream()`, `of()`, `range()`
Remove elements	`filter()`
Remove duplicates	`distinct()`
Take first few	`limit()`
Skip first few	`skip()`
Convert each element	`map()`
Merge nested lists	`flatMap()`
Sort elements	`sorted()`
Count elements	`count()`
Find smallest	`min()`
Find largest	`max()`
Add all numbers	`sum()`
Get average	`average()`
Find any match	`anyMatch()`
Get first element	`findFirst()`
Save as list	`collect(toList())`
Save as map	`collect(toMap())`

Group by key	`collect(groupingBy())`
Join strings	`collect(joining())`
Loop through	`forEach()`
Debug pipeline	`peek()`
Speed up processing	`parallelStream()`

---

10. METHOD CATEGORY MEMORY TRICK

```
CREATE   →  FILTER   →  TRANSFORM   →  COLLECT
──────────────────────────────────────────────

of()        filter()      map()          collect()
stream()    distinct()    flatMap()      toList()
range()     limit()       sorted()       toMap()
generate()  skip()        peek()         groupingBy()
iterate()   takeWhile()   boxed()        joining()
concat()    dropWhile()   mapToInt()     summingInt()
empty()                               →  summarizingInt()
```

Pro Tip: Read Method Names Like English

```
// Reads like: "From list, keep active items, get their prices, collect to list"
list.stream().filter(Item::isActive).map(Item::getPrice).collect(toList());

// Reads like: "From employees, group by department, count each group"
employees.stream().collect(groupingBy(Employee::getDepartment, counting()));

// Reads like: "From numbers, find first even number or return -1"
numbers.stream().filter(n -> n % 2 == 0).findFirst().orElse(-1);
```

JAVA 8 STREAMS CHEAT SHEET — Quick Reference

When to Use What — Decision Guide

---

1. STREAM CREATION

```
// When: You have data and need to start streaming
Stream.of("a", "b", "c")                    // Individual elements
Arrays.stream(new int[]{1, 2, 3})           // From array
list.stream()                                // From Collection
Stream.iterate(0, n -> n + 2)               // Infinite sequence
Stream.generate(Math::random)               // Infinite supplier
IntStream.range(1, 100)                     // Range of numbers
"string".chars()                            // String to IntStream
Stream.empty()                              // Empty stream
Stream.concat(stream1, stream2)             // Combine two streams
```

---

2. INTERMEDIATE OPERATIONS (Lazy)

Filtering

```
// When: Remove elements that don't match condition
.filter(s -> s.startsWith("a"))             // Keep only 'a' strings
.distinct()                                  // Remove duplicates
.limit(10)                                   // Take first 10
.skip(5)                                     // Skip first 5
.takeWhile(s -> s.length() > 2)             // Java 9+: Take while condition true
.dropWhile(s -> s.length() > 2)             // Java 9+: Drop while condition true
```

Transforming

```
// When: Convert each element to something else
.map(String::toUpperCase)                   // Transform each element
.mapToInt(String::length)                   // To IntStream
.mapToDouble(Employee::getSalary)           // To DoubleStream
.flatMap(line -> Arrays.stream(line.split(" ")))  // One-to-many transformation
.boxed()                                     // IntStream → Stream<Integer>
```

Sorting

```
// When: Need ordered results
.sorted()                                    // Natural order
.sorted(Comparator.reverseOrder())          // Reverse natural order
.sorted(Comparator.comparing(Employee::getSalary))  // By field
.sorted(Comparator.comparing(Employee::getSalary).reversed())  // Descending
.sorted(Comparator.comparing(Employee::getSalary).thenComparing(Employee::getName)
)
```

Peeking (Debugging)

```
// When: Debug intermediate values (side effect - be careful!)
.peek(System.out::println)                  // Print each element (debug only)
.peek(e -> log.info("Processing: {}", e))   // Logging
```

---

3. TERMINAL OPERATIONS (Execute Stream)

Aggregation

```
// When: Need single value from stream
.count()                                     // Number of elements
.min(Comparator.naturalOrder())             // Minimum element
.max(Comparator.comparing(Employee::getSalary))  // Maximum element
.sum()                                       // IntStream/DoubleStream sum
.average()                                   // IntStream/DoubleStream average
.reduce(0, Integer::sum)                    // Combine to single value
.reduce((a, b) -> a + b)                    // Without identity
.findFirst()                                 // First element (Optional)
.findAny()                                   // Any element (parallel friendly)
```

Collection

```
// When: Need to collect results into container
.collect(Collectors.toList())               // To ArrayList
.collect(Collectors.toSet())                // To HashSet
.collect(Collectors.toCollection(LinkedList::new))  // Specific collection
.collect(Collectors.toUnmodifiableList())   // Java 10+: Immutable list
.toList()                                    // Java 16+: Simplified
```

Iteration

```
// When: Perform action for each element
.forEach(System.out::println)                // Print each (terminal)
.forEachOrdered(System.out::println)         // Preserve order in parallel
```

Matching

```
// When: Check conditions on stream
.anyMatch(s -> s.length() > 5)              // Any element matches
.allMatch(s -> s.length() > 0)              // All elements match
.noneMatch(s -> s.isEmpty())                // No element matches
```

---

4. COLLECTORS — The Most Important

Grouping

```
// When: Group elements by some key
.collect(Collectors.groupingBy(Employee::getDepartment))
.collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()))
.collect(Collectors.groupingBy(Employee::getDepartment,
         Collectors.mapping(Employee::getName, Collectors.toList())))

// When: Partition into true/false
.collect(Collectors.partitioningBy(e -> e.getSalary() > 50000))
```

Joining

```
// When: Combine strings with delimiter
.collect(Collectors.joining(", "))           // "a, b, c"
.collect(Collectors.joining(", ", "[", "]")) // "[a, b, c]"
```

Summarizing

```
// When: Need multiple stats at once
.collect(Collectors.summarizingInt(Employee::getAge))
// Returns: IntSummaryStatistics{count=5, sum=150, min=25, max=40, average=30.0}
```

Mapping & Reducing

```
// When: Transform before collecting
.collect(Collectors.mapping(Employee::getName, Collectors.toList()))

// When: Custom reduction
.collect(Collectors.reducing(0, Employee::getSalary, Integer::sum))

// When: FlatMap during collect
.collect(Collectors.flatMapping(e -> e.getPhones().stream(), Collectors.toList()))
```

To Map

```
// When: Convert list to map
.collect(Collectors.toMap(Employee::getId, Function.identity()))
.collect(Collectors.toMap(Employee::getId, Employee::getName))
.collect(Collectors.toMap(Employee::getId, Function.identity(), (e1, e2) -> e1))
.collect(Collectors.toMap(Employee::getId, Function.identity(), (e1, e2) -> e1,
LinkedHashMap::new))
```

---

5. PRIMITIVE STREAMS (Performance)

IntStream

```
// When: Working with primitives (faster, saves memory)
IntStream.range(1, 100)                      // 1 to 99
IntStream.rangeClosed(1, 100)               // 1 to 100
IntStream.of(1, 2, 3)                       // From values
Arrays.stream(intArray)                      // From int[]

// Operations
.sum() .average() .min() .max() .count()
.sorted() .distinct() .filter(n -> n > 5)
.boxed()                                     // To Stream<Integer>
.mapToObj(i -> "Number: " + i)              // To object stream
```

DoubleStream & LongStream

```
DoubleStream.of(1.0, 2.0, 3.0)
LongStream.range(1, 100)
```

---

6. COMMON PATTERNS (Copy-Paste Ready)

Pattern 1: Filter Nulls

```
list.stream()
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
```

Pattern 2: Find Duplicates

```
Set<String> seen = new HashSet<>();
list.stream()
    .filter(e -> !seen.add(e))
    .collect(Collectors.toSet());
```

Pattern 3: Most Frequent Element

```
list.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey)
    .orElse(null);
```

Pattern 4: Chunk/Batch Processing

```
IntStream.range(0, list.size())
    .boxed()
    .collect(Collectors.groupingBy(i -> i / batchSize))
    .values()
    .stream()
    .map(indices -> indices.stream().map(list::get).collect(Collectors.toList()))
    .forEach(batch -> process(batch));
```

Pattern 5: Cartesian Product

```
list1.stream()
    .flatMap(a -> list2.stream().map(b -> new Pair<>(a, b)))
    .collect(Collectors.toList());
```

Pattern 6: Nested Collection Flatten

```
List<List<String>> nested = ...;
nested.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

Pattern 7: Custom Collector (Top N by Value)

```
list.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        l -> l.stream().limit(5).collect(Collectors.toList())
    ));
```

Pattern 8: Pagination Simulation

```
IntStream.range(0, (list.size() + pageSize - 1) / pageSize)
    .mapToObj(page -> list.stream()
        .skip(page * pageSize)
        .limit(pageSize)
        .collect(Collectors.toList()))
    .collect(Collectors.toList());
```

---

7. PERFORMANCE TIPS (Quick Memory)

When	Use	Avoid
Large dataset, multi-core	`.parallelStream()`	`.stream()`
Small dataset	`.stream()`	`.parallelStream()` (overhead)
Simple operations	`.map().filter()`	Custom Spliterator
Need order	`.forEachOrdered()`	`.forEach()`
Infinite streams	`.limit()` or `.findFirst()`	No limit (memory crash)
Primitive operations	`IntStream`, `LongStream`	`Stream<Integer>` (boxing)

---

8. COMMON MISTAKES TO AVOID

```
// ❌ WRONG: Reusing stream
Stream<String> stream = list.stream();
stream.forEach(System.out::println);
stream.count();  // IllegalStateException!

// ✅ RIGHT: Create new stream each time

// ❌ WRONG: Modifying source while streaming
list.stream().filter(e -> list.remove(e));  // ConcurrentModificationException

// ✅ RIGHT: Collect and modify separately

// ❌ WRONG: Using parallel on non-thread-safe collections
ArrayList<String> list = new ArrayList<>();
list.parallelStream().forEach(s -> list.add(s));  // Race condition!

// ✅ RIGHT: Use Concurrent collections or collect results

// ❌ WRONG: Calling terminal operation multiple times
Stream<Integer> stream = Stream.of(1,2,3);
stream.count();
stream.collect(Collectors.toList());  // Already consumed!

// ✅ RIGHT: Chain everything before terminal operation
```

---

9. QUICK VISUAL REFERENCE

```
Source → Intermediate → Intermediate → Terminal → Result

list.stream()   .filter()     .map()       .collect()
array.stream()  .distinct()   .flatMap()   .forEach()
Stream.of()     .sorted()     .limit()     .reduce()
                .peek()       .skip()      .findFirst()
```

---

10. ONE-LINE SOLUTIONS (Interview Gems)

```
// Remove duplicates
list.stream().distinct().collect(Collectors.toList());

// Convert list to map
list.stream().collect(Collectors.toMap(Item::getId, Function.identity()));

// Sum of salaries
employees.stream().mapToDouble(Employee::getSalary).sum();

// Group by department
employees.stream().collect(Collectors.groupingBy(Employee::getDepartment));

// Filter and transform
list.stream().filter(x -> x >
5).map(String::valueOf).collect(Collectors.toList());

// Check if any matches condition
list.stream().anyMatch(x -> x > 100);

// First element or default
list.stream().findFirst().orElse(defaultValue);

// Join strings with delimiter
list.stream().collect(Collectors.joining(","));

// Average of integers
ints.stream().mapToInt(i -> i).average().orElse(0);

// Sort descending
list.stream().sorted(Comparator.reverseOrder()).collect(Collectors.toList());

// Max by field
employees.stream().max(Comparator.comparing(Employee::getSalary)).get();

// Square and sum
numbers.stream().mapToInt(n -> n * n).sum();

// Partition by condition
list.stream().collect(Collectors.partitioningBy(x -> x > 10));
```

Memory Aid: COMMAND Pattern

```
Create  →  Filter  →  Transform  →  Collect/Save
.of()       .filter()   .map()        .collect()
.range()    .distinct() .flatMap()    .toList()
.stream()   .limit()    .sorted()     .toSet()
```

Real Example:

```
List<Employee> seniorDevelopers = employees.stream()
    .filter(e -> e.getAge() > 30)
    .filter(e -> "IT".equals(e.getDepartment()))
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(5)
    .collect(Collectors.toList());
```

Pro Tip: Method References vs Lambdas

```
// Lambda (verbose)
.map(s -> s.toUpperCase())

// Method Reference (clean)
.map(String::toUpperCase)

// Common method references:
String::toUpperCase      // instance method
String::length           // instance method
Employee::getName        // getter
Math::max                // static method
System.out::println      // specific instance
this::processMethod      // this instance
```

Easy, Medium, Hard with Stream Solutions

EASY LEVEL

Problem 1: Find Duplicate Characters in String

Stream-Based Solution

```
import java.util.*;
import java.util.stream.*;
import java.util.function.Function;

public class DuplicateCharacters {

    // Method 1: Using Streams with Collections.frequency
    public static Set<Character> findDuplicatesStreams(String str) {
        if (str == null || str.isEmpty()) return Collections.emptySet();

        return str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() > 1)
            .map(Map.Entry::getKey)
            .collect(Collectors.toSet());
    }

    // Method 2: Without streams (traditional)
    public static Set<Character> findDuplicatesTraditional(String str) {
        Set<Character> duplicates = new HashSet<>();
        Set<Character> seen = new HashSet<>();

        for (char c : str.toCharArray()) {
            if (!seen.add(c)) {
                duplicates.add(c);
            }
        }
        return duplicates;
    }

    // Method 3: Find duplicates with their counts
    public static Map<Character, Long> findDuplicateCounts(String str) {
        return str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(),
Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() > 1)
            .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue));
```

```
    }

    // Method 4: First non-repeating character using streams
    public static Character firstNonRepeatingChar(String str) {
        return str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(),
LinkedHashMap::new, Collectors.counting()))
            .entrySet().stream()
            .filter(entry -> entry.getValue() == 1)
            .map(Map.Entry::getKey)
            .findFirst()
            .orElse(null);
    }

    public static void main(String[] args) {
        String input = "programming";
        System.out.println("Duplicate characters: " +
findDuplicatesStreams(input));
        System.out.println("Duplicate counts: " + findDuplicateCounts(input));
        System.out.println("First non-repeating: " +
firstNonRepeatingChar(input));
        // Output: Duplicate characters: [r, g, m]
    }
}
```

Problem 2: Fibonacci Series

Multiple Solutions

```
import java.util.stream.*;
import java.util.*;

public class FibonacciGenerator {

    // Method 1: Using Stream.iterate (Infinite stream)
    public static List<Long> fibonacciStreams(int n) {
        return Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] +
f[1]})
            .limit(n)
            .map(f -> f[0])
            .collect(Collectors.toList());
    }

    // Method 2: Using IntStream.generate
    public static List<Integer> fibonacciGenerate(int n) {
        return IntStream.generate(new Supplier<Integer>() {
            private int prev = 0;
            private int current = 1;

            @Override
            public Integer get() {
                int next = prev;
                int temp = prev + current;
                prev = current;
                current = temp;
                return next;
            }
        }).limit(n)
          .boxed()
          .collect(Collectors.toList());
    }

    // Method 3: Recursive (classic, but inefficient for large n)
    public static long fibonacciRecursive(int n) {
        if (n <= 1) return n;
        return fibonacciRecursive(n - 1) + fibonacciRecursive(n - 2);
    }

    // Method 4: Memoized recursion (efficient)
    private static Map<Integer, Long> memo = new HashMap<>();
```

```
    public static long fibonacciMemoized(int n) {
        if (n <= 1) return n;
        return memo.computeIfAbsent(n,
            k -> fibonacciMemoized(n - 1) + fibonacciMemoized(n - 2));
    }

    // Method 5: Iterative (most efficient)
    public static long fibonacciIterative(int n) {
        if (n <= 1) return n;
        long prev = 0, current = 1;
        for (int i = 2; i <= n; i++) {
            long next = prev + current;
            prev = current;
            current = next;
        }
        return current;
    }

    // Method 6: Find first fibonacci number greater than threshold
    public static long firstFibonacciAbove(long threshold) {
        return Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] +
f[1]})
            .filter(f -> f[0] > threshold)
            .findFirst()
            .map(f -> f[0])
            .orElse(-1L);
    }

    // Method 7: Check if number is fibonacci
    public static boolean isFibonacci(long num) {
        // A number is Fibonacci if (5*n^2 + 4) or (5*n^2 - 4) is perfect square
        long square1 = 5 * num * num + 4;
        long square2 = 5 * num * num - 4;
        return isPerfectSquare(square1) || isPerfectSquare(square2);
    }

    private static boolean isPerfectSquare(long x) {
        long s = (long) Math.sqrt(x);
        return s * s == x;
    }

    public static void main(String[] args) {
        System.out.println("First 10 Fibonacci: " + fibonacciStreams(10));
```

```
        // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

        System.out.println("First > 100: " + firstFibonacciAbove(100));
        // 144
    }
}
```

Problem 3: Palindrome Check

```
public class PalindromeChecker {

    // Stream-based solution
    public static boolean isPalindromeStream(String str) {
        String cleaned = str.toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
        return IntStream.range(0, cleaned.length() / 2)
            .allMatch(i -> cleaned.charAt(i) == cleaned.charAt(cleaned.length() -
1 - i));
    }

    // Using StringBuilder (simplest)
    public static boolean isPalindromeSimple(String str) {
        String cleaned = str.toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
        return new StringBuilder(cleaned).reverse().toString().equals(cleaned);
    }

    // Two-pointer technique
    public static boolean isPalindromeTwoPointer(String str) {
        String cleaned = str.toLowerCase().replaceAll("[^a-zA-Z0-9]", "");
        int left = 0, right = cleaned.length() - 1;
        while (left < right) {
            if (cleaned.charAt(left++) != cleaned.charAt(right--)) {
                return false;
            }
        }
        return true;
    }
}
```

MEDIUM LEVEL

Problem 4: Second Highest Salary (Classic Stream Problem)

Employee Class and Solutions

```
import java.util.*;
import java.util.stream.*;

class Employee {
    private String name;
    private double salary;
    private String department;
    private int age;

    // Constructor, getters, toString
    public Employee(String name, double salary, String department, int age) {
        this.name = name;
        this.salary = salary;
        this.department = department;
        this.age = age;
    }

    public double getSalary() { return salary; }
    public String getName() { return name; }
    public String getDepartment() { return department; }
    public int getAge() { return age; }

    @Override
    public String toString() {
        return String.format("%s - $%.2f - %s - %d", name, salary, department,
age);
    }
}

public class SalaryAnalyzer {

    // Method 1: Second highest salary overall
    public static Optional<Double> secondHighestSalary(List<Employee> employees) {
        return employees.stream()
            .map(Employee::getSalary)
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(1)
            .findFirst();
    }

    // Method 2: Second highest employee (full object)
```

```
    public static Optional<Employee> secondHighestEmployee(List<Employee>
employees) {
        return employees.stream()
            .sorted(Comparator.comparing(Employee::getSalary).reversed())
            .distinct()
            .skip(1)
            .findFirst();
    }

    // Method 3: Second highest salary per department
    public static Map<String, Optional<Double>>
secondHighestPerDepartment(List<Employee> employees) {
        return employees.stream()
            .collect(Collectors.groupingBy(
                Employee::getDepartment,
                Collectors.collectingAndThen(
                    Collectors.mapping(Employee::getSalary, Collectors.toList()),
                    salaries -> salaries.stream()
                        .distinct()
                        .sorted(Comparator.reverseOrder())
                        .skip(1)
                        .findFirst()
                )
            ));
    }

    // Method 4: Nth highest salary (generic)
    public static Optional<Double> nthHighestSalary(List<Employee> employees, int
n) {
        if (n <= 0) return Optional.empty();

        return employees.stream()
            .map(Employee::getSalary)
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(n - 1)
            .findFirst();
    }

    // Method 5: Second highest with duplicates handled
    public static Optional<Employee> secondHighestByRank(List<Employee> employees)
{
        return employees.stream()
```

```
            .collect(Collectors.groupingBy(Employee::getSalary))
            .entrySet().stream()
            .sorted(Map.Entry.<Double, List<Employee>>comparingByKey().reversed())
            .skip(1)
            .findFirst()
            .map(Map.Entry::getValue)
            .flatMap(list -> list.stream().findFirst());
    }

    // Method 6: Top 3 salaries across all departments
    public static List<Employee> topThreeSalaries(List<Employee> employees) {
        return employees.stream()
            .sorted(Comparator.comparing(Employee::getSalary).reversed())
            .limit(3)
            .collect(Collectors.toList());
    }

    public static void main(String[] args) {
        List<Employee> employees = Arrays.asList(
            new Employee("John", 100000, "IT", 30),
            new Employee("Jane", 95000, "HR", 28),
            new Employee("Bob", 100000, "IT", 35),  // Duplicate salary
            new Employee("Alice", 90000, "HR", 32),
            new Employee("Charlie", 110000, "IT", 40),
            new Employee("Diana", 85000, "Finance", 29)
        );

        secondHighestSalary(employees).ifPresent(s ->
            System.out.println("Second highest salary: $" + s));
        // Second highest salary: $100000 (not 95000 because duplicate handling)

        secondHighestPerDepartment(employees).forEach((dept, salary) ->
            System.out.println(dept + " second highest: $" + salary.orElse(0.0)));

        // IT second highest: $100000.0
        // HR second highest: $90000.0
        // Finance second highest: $0.0 (only one employee)
    }
}
```

Problem 5: Find All Permutations of String (Medium/Hard)

```
public class StringPermutations {

    // Method 1: Generate all distinct permutations using Streams approach
    public static Set<String> permutationsStream(String str) {
        if (str == null || str.isEmpty()) {
            return new HashSet<>();
        }

        return permutationsHelper(str, "")
            .collect(Collectors.toSet());
    }

    private static Stream<String> permutationsHelper(String remaining, String
prefix) {
        if (remaining.isEmpty()) {
            return Stream.of(prefix);
        }

        return IntStream.range(0, remaining.length())
            .boxed()
            .flatMap(i -> {
                char ch = remaining.charAt(i);
                String newRemaining = remaining.substring(0, i) +
remaining.substring(i + 1);
                return permutationsHelper(newRemaining, prefix + ch);
            });
    }

    // Method 2: Backtracking approach (more efficient)
    public static List<String> permutationsBacktrack(String str) {
        List<String> result = new ArrayList<>();
        char[] chars = str.toCharArray();
        backtrack(chars, 0, result);
        return result;
    }

    private static void backtrack(char[] chars, int index, List<String> result) {
        if (index == chars.length - 1) {
            result.add(new String(chars));
            return;
        }
```

```
        for (int i = index; i < chars.length; i++) {
            swap(chars, index, i);
            backtrack(chars, index + 1, result);
            swap(chars, index, i); // backtrack
        }
    }

    private static void swap(char[] chars, int i, int j) {
        char temp = chars[i];
        chars[i] = chars[j];
        chars[j] = temp;
    }

    // Method 3: Unique permutations only (handle duplicates)
    public static List<String> uniquePermutations(String str) {
        List<String> result = new ArrayList<>();
        char[] chars = str.toCharArray();
        Arrays.sort(chars); // Sort to handle duplicates
        boolean[] used = new boolean[chars.length];
        backtrackUnique(chars, used, new StringBuilder(), result);
        return result;
    }

    private static void backtrackUnique(char[] chars, boolean[] used,
                                        StringBuilder current, List<String>
result) {
        if (current.length() == chars.length) {
            result.add(current.toString());
            return;
        }

        for (int i = 0; i < chars.length; i++) {
            if (used[i]) continue;
            // Skip duplicates
            if (i > 0 && chars[i] == chars[i - 1] && !used[i - 1]) continue;

            used[i] = true;
            current.append(chars[i]);
            backtrackUnique(chars, used, current, result);
            current.deleteCharAt(current.length() - 1);
            used[i] = false;
        }
    }
```

```
    public static void main(String[] args) {
        System.out.println("Permutations of 'abc': " +
permutationsBacktrack("abc"));
        // [abc, acb, bac, bca, cab, cba]

        System.out.println("Unique permutations of 'aab': " +
uniquePermutations("aab"));
        // [aab, aba, baa]
    }
}
```

Problem 6: Group Anagrams

```
public class AnagramGrouper {

    // Stream-based solution - group anagrams
    public static Map<String, List<String>> groupAnagrams(String[] words) {
        return Arrays.stream(words)
            .collect(Collectors.groupingBy(
                word -> word.chars()
                    .sorted()
                    .collect(StringBuilder::new,
                             StringBuilder::appendCodePoint,
                             StringBuilder::append)
                    .toString()
            ));
    }

    // More efficient with char array sorting
    public static Map<String, List<String>> groupAnagramsEfficient(String[] words)
{
        return Arrays.stream(words)
            .collect(Collectors.groupingBy(word -> {
                char[] chars = word.toCharArray();
                Arrays.sort(chars);
                return new String(chars);
            }));
    }

    // Find all anagram groups and return only those with size > 1
    public static List<List<String>> findAnagramGroups(String[] words) {
        return new ArrayList<>(groupAnagramsEfficient(words).values().stream()
            .filter(group -> group.size() > 1)
            .collect(Collectors.toList()));
    }

    public static void main(String[] args) {
        String[] words = {"eat", "tea", "tan", "ate", "nat", "bat"};
        System.out.println(groupAnagrams(words));
        // {aet=[eat, tea, ate], ant=[tan, nat], abt=[bat]}
    }
}
```

HARD LEVEL

Problem 7: Tree Problem — Lowest Common Ancestor in Binary Tree

```java
import java.util.*;
import java.util.stream.*;

// Binary Tree Node
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
    }

    TreeNode(int val, TreeNode left, TreeNode right) {
        this.val = val;
        this.left = left;
        this.right = right;
    }

    @Override
    public String toString() {
        return String.valueOf(val);
    }
}

public class BinaryTreeLCA {

    // Problem 1: Lowest Common Ancestor (LCA) in Binary Tree
    public static TreeNode findLCA(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null || root == p || root == q) {
            return root;
        }

        TreeNode left = findLCA(root.left, p, q);
        TreeNode right = findLCA(root.right, p, q);

        if (left != null && right != null) {
            return root; // Found LCA
        }

        return left != null ? left : right;
    }
```

```
    // Problem 2: LCA in Binary Search Tree (more efficient)
    public static TreeNode findLCAInBST(TreeNode root, TreeNode p, TreeNode q) {
        if (root == null) return null;

        // If both values are smaller, go left
        if (p.val < root.val && q.val < root.val) {
            return findLCAInBST(root.left, p, q);
        }

        // If both values are larger, go right
        if (p.val > root.val && q.val > root.val) {
            return findLCAInBST(root.right, p, q);
        }

        // One on each side, current node is LCA
        return root;
    }

    // Problem 3: Find path from root to node (using streams)
    public static List<TreeNode> findPath(TreeNode root, TreeNode target) {
        List<TreeNode> path = new ArrayList<>();
        findPathHelper(root, target, path);
        return path;
    }

    private static boolean findPathHelper(TreeNode root, TreeNode target,
List<TreeNode> path) {
        if (root == null) return false;

        path.add(root);

        if (root == target) return true;

        if (findPathHelper(root.left, target, path) || findPathHelper(root.right,
target, path)) {
            return true;
        }

        path.remove(path.size() - 1);
        return false;
    }
```

```
    // Problem 4: Find distance between two nodes
    public static int findDistance(TreeNode root, TreeNode p, TreeNode q) {
        TreeNode lca = findLCA(root, p, q);
        int distP = findLevel(lca, p, 0);
        int distQ = findLevel(lca, q, 0);
        return distP + distQ;
    }

    private static int findLevel(TreeNode root, TreeNode target, int level) {
        if (root == null) return -1;
        if (root == target) return level;

        int left = findLevel(root.left, target, level + 1);
        return left != -1 ? left : findLevel(root.right, target, level + 1);
    }

    // Problem 5: Get all nodes at distance K from target
    public static List<TreeNode> nodesAtDistanceK(TreeNode root, TreeNode target,
int k) {
        List<TreeNode> result = new ArrayList<>();
        Map<TreeNode, TreeNode> parentMap = new HashMap<>();
        buildParentMap(root, null, parentMap);

        Queue<TreeNode> queue = new LinkedList<>();
        Set<TreeNode> visited = new HashSet<>();

        queue.offer(target);
        visited.add(target);
        int distance = 0;

        while (!queue.isEmpty() && distance < k) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();

                // Check left child
                if (node.left != null && !visited.contains(node.left)) {
                    visited.add(node.left);
                    queue.offer(node.left);
                }

                // Check right child
                if (node.right != null && !visited.contains(node.right)) {
```

```
                    visited.add(node.right);
                    queue.offer(node.right);
                }

                // Check parent
                TreeNode parent = parentMap.get(node);
                if (parent != null && !visited.contains(parent)) {
                    visited.add(parent);
                    queue.offer(parent);
                }
            }
            distance++;
        }

        result.addAll(queue);
        return result;
    }

    private static void buildParentMap(TreeNode root, TreeNode parent,
Map<TreeNode, TreeNode> map) {
        if (root == null) return;
        map.put(root, parent);
        buildParentMap(root.left, root, map);
        buildParentMap(root.right, root, map);
    }

    // Problem 6: Binary Tree Level Order Traversal (using streams)
    public static List<List<Integer>> levelOrderTraversal(TreeNode root) {
        if (root == null) return Collections.emptyList();

        List<List<Integer>> result = new ArrayList<>();
        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            int levelSize = queue.size();
            List<Integer> level = new ArrayList<>();

            for (int i = 0; i < levelSize; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) queue.offer(node.left);
                if (node.right != null) queue.offer(node.right);
```

```
            }
            result.add(level);
        }
        return result;
    }

    public static void main(String[] args) {
        // Build binary tree:
        //        3
        //       / \
        //      5   1
        //     / \ / \
        //    6  2 0 8
        //      / \
        //     7  4

        TreeNode root = new TreeNode(3);
        TreeNode node5 = new TreeNode(5);
        TreeNode node1 = new TreeNode(1);
        TreeNode node6 = new TreeNode(6);
        TreeNode node2 = new TreeNode(2);
        TreeNode node0 = new TreeNode(0);
        TreeNode node8 = new TreeNode(8);
        TreeNode node7 = new TreeNode(7);
        TreeNode node4 = new TreeNode(4);

        root.left = node5;
        root.right = node1;
        node5.left = node6;
        node5.right = node2;
        node1.left = node0;
        node1.right = node8;
        node2.left = node7;
        node2.right = node4;

        System.out.println("LCA of 5 and 1: " + findLCA(root, node5, node1).val);
        // Output: 3

        System.out.println("Distance between 7 and 4: " + findDistance(root,
node7, node4));
        // Output: 4 (7→2→5→3→1? Actually 7→2→5→3→1→0? Let's compute properly)

        System.out.println("Level order: " + levelOrderTraversal(root));
```

```
        // [[3], [5,1], [6,2,0,8], [7,4]]
    }
}
```

Problem 8: Graph Problem — Clone Graph & Detect Cycle

```java
import java.util.*;

// Graph Node
class GraphNode {
    int val;
    List<GraphNode> neighbors;

    GraphNode(int val) {
        this.val = val;
        this.neighbors = new ArrayList<>();
    }

    GraphNode(int val, List<GraphNode> neighbors) {
        this.val = val;
        this.neighbors = neighbors;
    }

    @Override
    public String toString() {
        return String.valueOf(val);
    }
}

public class GraphProblems {

    // Problem 1: Clone Graph (Deep Copy using BFS)
    public static GraphNode cloneGraph(GraphNode node) {
        if (node == null) return null;

        Map<GraphNode, GraphNode> visited = new HashMap<>();
        Queue<GraphNode> queue = new LinkedList<>();

        visited.put(node, new GraphNode(node.val));
        queue.offer(node);

        while (!queue.isEmpty()) {
            GraphNode current = queue.poll();

            for (GraphNode neighbor : current.neighbors) {
                if (!visited.containsKey(neighbor)) {
                    visited.put(neighbor, new GraphNode(neighbor.val));
                    queue.offer(neighbor);
```

```
                }
                visited.get(current).neighbors.add(visited.get(neighbor));
            }
        }

        return visited.get(node);
    }

    // Problem 2: Detect Cycle in Directed Graph (DFS)
    public static boolean hasCycleDirected(List<GraphNode> graph) {
        Set<GraphNode> visited = new HashSet<>();
        Set<GraphNode> recursionStack = new HashSet<>();

        for (GraphNode node : graph) {
            if (detectCycleDFSUtil(node, visited, recursionStack)) {
                return true;
            }
        }
        return false;
    }

    private static boolean detectCycleDFSUtil(GraphNode node, Set<GraphNode>
visited,
                                              Set<GraphNode> recursionStack) {
        if (recursionStack.contains(node)) return true;
        if (visited.contains(node)) return false;

        visited.add(node);
        recursionStack.add(node);

        for (GraphNode neighbor : node.neighbors) {
            if (detectCycleDFSUtil(neighbor, visited, recursionStack)) {
                return true;
            }
        }

        recursionStack.remove(node);
        return false;
    }

    // Problem 3: Detect Cycle in Undirected Graph (Union-Find)
    public static boolean hasCycleUndirected(List<Edge> edges, int vertices) {
        int[] parent = new int[vertices];
```

```
        for (int i = 0; i < vertices; i++) {
            parent[i] = i;
        }

        for (Edge edge : edges) {
            int root1 = find(parent, edge.src);
            int root2 = find(parent, edge.dest);

            if (root1 == root2) {
                return true; // Cycle detected
            }

            union(parent, root1, root2);
        }
        return false;
    }

    private static int find(int[] parent, int x) {
        if (parent[x] != x) {
            parent[x] = find(parent, parent[x]); // Path compression
        }
        return parent[x];
    }

    private static void union(int[] parent, int x, int y) {
        int rootX = find(parent, x);
        int rootY = find(parent, y);
        if (rootX != rootY) {
            parent[rootY] = rootX;
        }
    }

    static class Edge {
        int src, dest;
        Edge(int src, int dest) {
            this.src = src;
            this.dest = dest;
        }
    }

    // Problem 4: Find if path exists between two nodes (BFS)
    public static boolean hasPath(GraphNode start, GraphNode end) {
        if (start == end) return true;
```

```
        Set<GraphNode> visited = new HashSet<>();
        Queue<GraphNode> queue = new LinkedList<>();

        visited.add(start);
        queue.offer(start);

        while (!queue.isEmpty()) {
            GraphNode current = queue.poll();

            for (GraphNode neighbor : current.neighbors) {
                if (neighbor == end) return true;
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    queue.offer(neighbor);
                }
            }
        }
        return false;
    }

    // Problem 5: Topological Sort (Kahn's Algorithm)
    public static List<Integer> topologicalSort(List<GraphNode> graph, int
vertices) {
        int[] inDegree = new int[vertices];

        // Calculate in-degree for each node
        for (GraphNode node : graph) {
            for (GraphNode neighbor : node.neighbors) {
                inDegree[neighbor.val]++;
            }
        }

        // Add all nodes with 0 in-degree to queue
        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < vertices; i++) {
            if (inDegree[i] == 0) {
                queue.offer(i);
            }
        }

        List<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
```

```
            int current = queue.poll();
            result.add(current);

            // Decrease in-degree of neighbors
            for (GraphNode neighbor : graph.get(current).neighbors) {
                inDegree[neighbor.val]--;
                if (inDegree[neighbor.val] == 0) {
                    queue.offer(neighbor.val);
                }
            }
        }

        return result.size() == vertices ? result : new ArrayList<>(); // Cycle
detected
    }

    // Problem 6: Find shortest path in unweighted graph (BFS)
    public static List<GraphNode> shortestPath(GraphNode start, GraphNode end) {
        if (start == end) return List.of(start);

        Map<GraphNode, GraphNode> parent = new HashMap<>();
        Queue<GraphNode> queue = new LinkedList<>();

        parent.put(start, null);
        queue.offer(start);

        while (!queue.isEmpty()) {
            GraphNode current = queue.poll();

            for (GraphNode neighbor : current.neighbors) {
                if (!parent.containsKey(neighbor)) {
                    parent.put(neighbor, current);
                    if (neighbor == end) {
                        return reconstructPath(parent, end);
                    }
                    queue.offer(neighbor);
                }
            }
        }

        return Collections.emptyList(); // No path found
    }
```

```
    private static List<GraphNode> reconstructPath(Map<GraphNode, GraphNode>
parent, GraphNode end) {
        List<GraphNode> path = new ArrayList<>();
        GraphNode current = end;

        while (current != null) {
            path.add(0, current);
            current = parent.get(current);
        }

        return path;
    }

    public static void main(String[] args) {
        // Create graph: 0->1, 0->2, 1->2, 2->0, 2->3, 3->3
        GraphNode node0 = new GraphNode(0);
        GraphNode node1 = new GraphNode(1);
        GraphNode node2 = new GraphNode(2);
        GraphNode node3 = new GraphNode(3);

        node0.neighbors = Arrays.asList(node1, node2);
        node1.neighbors = Arrays.asList(node2);
        node2.neighbors = Arrays.asList(node0, node3);
        node3.neighbors = Arrays.asList(node3);

        List<GraphNode> graph = Arrays.asList(node0, node1, node2, node3);

        System.out.println("Has cycle: " + hasCycleDirected(graph)); // true

        // Shortest path
        List<GraphNode> path = shortestPath(node0, node3);
        System.out.println("Shortest path 0→3: " + path);
        // [0, 2, 3]
    }
}
```

Problem 9: Backtracking — N-Queens Problem

```
import java.util.*;
import java.util.stream.IntStream;

public class NQueensProblem {

    // Problem: Place N queens on NxN chessboard so no two attack each other

    // Solution 1: Standard backtracking
    public static List<List<String>> solveNQueens(int n) {
        List<List<String>> solutions = new ArrayList<>();
        char[][] board = new char[n][n];

        for (int i = 0; i < n; i++) {
            Arrays.fill(board[i], '.');
        }

        backtrack(board, 0, solutions);
        return solutions;
    }

    private static void backtrack(char[][] board, int row, List<List<String>>
solutions) {
        if (row == board.length) {
            solutions.add(constructSolution(board));
            return;
        }

        for (int col = 0; col < board.length; col++) {
            if (isSafe(board, row, col)) {
                board[row][col] = 'Q';
                backtrack(board, row + 1, solutions);
                board[row][col] = '.'; // Undo
            }
        }
    }

    private static boolean isSafe(char[][] board, int row, int col) {
        // Check column
        for (int i = 0; i < row; i++) {
            if (board[i][col] == 'Q') return false;
        }
```

```
        // Check diagonal (top-left to bottom-right)
        for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
            if (board[i][j] == 'Q') return false;
        }

        // Check diagonal (top-right to bottom-left)
        for (int i = row - 1, j = col + 1; i >= 0 && j < board.length; i--, j++) {
            if (board[i][j] == 'Q') return false;
        }

        return true;
    }

    private static List<String> constructSolution(char[][] board) {
        return Arrays.stream(board)
            .map(String::new)
            .collect(Collectors.toList());
    }

    // Solution 2: Optimized with column and diagonal tracking
    public static List<List<String>> solveNQueensOptimized(int n) {
        List<List<String>> solutions = new ArrayList<>();
        int[] queens = new int[n]; // queens[row] = column
        boolean[] cols = new boolean[n];
        boolean[] diag1 = new boolean[2 * n - 1]; // row - col + n - 1
        boolean[] diag2 = new boolean[2 * n - 1]; // row + col

        backtrackOptimized(queens, cols, diag1, diag2, 0, n, solutions);
        return solutions;
    }

    private static void backtrackOptimized(int[] queens, boolean[] cols, boolean[]
diag1,
                                           boolean[] diag2, int row, int n,
                                           List<List<String>> solutions) {
        if (row == n) {
            solutions.add(constructSolutionOptimized(queens, n));
            return;
        }

        for (int col = 0; col < n; col++) {
            int d1 = row - col + n - 1;
            int d2 = row + col;
```

```
            if (!cols[col] && !diag1[d1] && !diag2[d2]) {
                queens[row] = col;
                cols[col] = true;
                diag1[d1] = true;
                diag2[d2] = true;

                backtrackOptimized(queens, cols, diag1, diag2, row + 1, n,
solutions);

                cols[col] = false;
                diag1[d1] = false;
                diag2[d2] = false;
            }
        }
    }

    private static List<String> constructSolutionOptimized(int[] queens, int n) {
        return IntStream.range(0, n)
            .mapToObj(row -> {
                char[] line = new char[n];
                Arrays.fill(line, '.');
                line[queens[row]] = 'Q';
                return new String(line);
            })
            .collect(Collectors.toList());
    }

    // Solution 3: Count total solutions (no board construction)
    public static int totalNQueens(int n) {
        int[] count = new int[]{0};
        boolean[] cols = new boolean[n];
        boolean[] diag1 = new boolean[2 * n - 1];
        boolean[] diag2 = new boolean[2 * n - 1];

        backtrackCount(cols, diag1, diag2, 0, n, count);
        return count[0];
    }

    private static void backtrackCount(boolean[] cols, boolean[] diag1, boolean[]
diag2,
                                       int row, int n, int[] count) {
        if (row == n) {
```

```
            count[0]++;
            return;
        }

        for (int col = 0; col < n; col++) {
            int d1 = row - col + n - 1;
            int d2 = row + col;

            if (!cols[col] && !diag1[d1] && !diag2[d2]) {
                cols[col] = true;
                diag1[d1] = true;
                diag2[d2] = true;

                backtrackCount(cols, diag1, diag2, row + 1, n, count);

                cols[col] = false;
                diag1[d1] = false;
                diag2[d2] = false;
            }
        }
    }

    // Solution 4: Stream-based solution verification (check if board is valid)
    public static boolean isValidNQueensSolution(List<String> board) {
        int n = board.size();
        List<int[]> queens = IntStream.range(0, n)
            .boxed()
            .flatMap(row -> IntStream.range(0, n)
                .filter(col -> board.get(row).charAt(col) == 'Q')
                .mapToObj(col -> new int[]{row, col}))
            .collect(Collectors.toList());

        if (queens.size() != n) return false;

        return queens.stream().noneMatch(q1 ->
            queens.stream().anyMatch(q2 ->
                q1 != q2 && (q1[0] == q2[0] || q1[1] == q2[1] ||
                Math.abs(q1[0] - q2[0]) == Math.abs(q1[1] - q2[1]))
            )
        );
    }

    public static void main(String[] args) {
```

```
        System.out.println("N-Queens Solutions for N=4:");
        List<List<String>> solutions = solveNQueens(4);
        solutions.forEach(solution -> {
            System.out.println("Solution:");
            solution.forEach(System.out::println);
            System.out.println();
        });

        System.out.println("Total solutions for N=8: " + totalNQueens(8));
        // Output: 92
    }
}
```

QUICK REFERENCE SUMMARY

Difficulty	Problem	Key Technique	Stream Usage
Easy	Duplicate Characters	GroupingBy, Filter	✅ Heavy
Easy	Fibonacci	Stream.iterate, Generate	✅ Heavy
Easy	Palindrome	IntStream.range, allMatch	✅ Heavy
Medium	Second Highest Salary	Sorting, Skip, Distinct	✅ Heavy
Medium	Permutations	Backtracking	❌ Minimal
Medium	Group Anagrams	GroupingBy, Sorting	✅ Heavy
Hard	LCA in Tree	DFS, Recursion	❌ None
Hard	Clone Graph	BFS, HashMap	❌ None
Hard	N-Queens	Backtracking	❌ None

One-Line Stream Gems (Bonus)

```java
// Reverse a string
String reversed = new StringBuilder(str).reverse().toString();

// Check if string contains only digits
boolean isNumeric = str.chars().allMatch(Character::isDigit);

// Find most frequent character
char mostFrequent = str.chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .get().getKey();

// Check if two strings are anagrams
boolean areAnagrams = str1.length() == str2.length() &&
    str1.chars().sorted().boxed().collect(Collectors.toList())
    .equals(str2.chars().sorted().boxed().collect(Collectors.toList()));

// Remove duplicates from list preserving order
List<Integer> distinct = list.stream()
    .distinct()
    .collect(Collectors.toList());

// Find common elements between two lists
List<Integer> common = list1.stream()
    .filter(list2::contains)
    .collect(Collectors.toList());
```